EXPERIMENT NO: 9 **Date:** _____

LIST OF FREQUENCIES

AIM

Write a program to create list of frequencies in non-increasing order using function

OBJECTIVES

- To build frequency maps using Python dictionaries.
- To extract dictionary values into lists and implement custom sorting logic.
- To arrange frequency counts in descending numerical order.

ALGORITHM

Step 1 :- Initialize frequency distribution analyzer program to count list occurrences and sort in descending order.

Step 2 :- Declare function frequency list accepting array parameter arr for calculating item occurrence counts.

Step 3 :- Return empty list immediately if input list arr contains no numerical elements.

Step 4 :- Iterate items in list arr, updating element counts inside dictionary frequency dict.

Step 5 :- Convert dictionary count values into linear list frequencies for sorting processing.

Step 6 :- Run outer loop over frequency list indices to begin descending bubble sort algorithm.

Step 7 :- Run inner loop comparing current frequency against subsequent elements in frequencies list.

Step 8 :- Swap elements if preceding frequency count is smaller than following frequency count.

Step 9 :- Return sorted frequency list from function and print result to standard output screen.

Step 10 :- Conclude execution of frequency counting script and release dictionary memory back to system.

PROGRAM

```
def frequency_list(arr):
    if not arr:
        return []
    frequency_dict = {}

    for item in arr:
        if item in frequency_dict:
            frequency_dict[item] += 1
        else:
            frequency_dict[item] = 1
    frequencies = list(frequency_dict.values())

    for i in range(len(frequencies)):
```

```
        for j in range(i + 1, len(frequencies)):
            if frequencies[i] < frequencies[j]:
                frequencies[i], frequencies[j] = frequencies[j], frequencies
    [i]

    return frequencies

sample_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
frequencies = frequency_list(sample_list)
print(f"List of frequencies in non-increasing order: {frequencies}")
```

SAMPLE OUTPUT

Terminal Output

```
List of frequencies in non-increasing order:  [3, 2, 2, 1, 1, 1, 1]
```

RESULT

The frequency list computation program in non-increasing order was tested and verified.

EXPERIMENT NO: 10 **Date:** _____

FREQUENCY OF CHARACTERS OF A FILE

AIM

Write a program to find Frequency of characters in a given file

OBJECTIVES

- To manage file I/O operations using Python context managers (with open).
- To handle file stream exceptions like FileNotFoundError.
- To count character occurrences across file text content.

ALGORITHM

Step 1 :- Initialize character frequency counting program to process and count character occurrences in a file.

Step 2 :- Create function character frequency taking filename argument to read file and count characters.

Step 3 :- Open file in read mode safely using open context manager and store content in text.

Step 4 :- Iterate through each character char in text, updating counts in dictionary frequency dict.

Step 5 :- Intercept FileNotFoundError exception using try except block if target file does not exist.

Step 6 :- Open file.txt in write mode, write string hello world, and close file handle.

Step 7 :- Invoke character frequency function with filename and print calculated frequency dictionary to console.

Step 8 :- Finish file operation execution context and release file stream resources back to operating system.

PROGRAM

```
def character_frequency(filename):
    try:
        with open(filename, 'r') as file:
            text = file.read()
        frequency_dict = {}
        for char in text:
            if char in frequency_dict:
                frequency_dict[char] += 1
            else:
                frequency_dict[char] = 1
        return frequency_dict
    except FileNotFoundError:
        print(f"The file {filename} does not exist.")
        return {}

```

```
filename = 'file.txt'
f = open(filename, 'w')
f.write("hello world")
f.close()

frequencies = character_frequency(filename)
print(f"Frequencies of characters in the file: {frequencies}")
```

SAMPLE OUTPUT

Terminal Output

```
Frequencies of characters in the file: 'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1,
'w': 1, 'r': 1, 'd': 1
```

RESULT

The file character frequency analyzer was implemented and output verified successfully.